

```
In [10]: from tqdm.auto import tqdm
import time
seconds = 90*60
for remaining_time in tqdm(range(seconds),total=seconds,desc='90 Minute Timer',unit=
    time.sleep(1)
print('Time is UP --> GG')
```

90 Minute Timer: 0%| | 0/5400 [00:00<?, ?sec/s]

KeyboardInterrupt

Traceback (most recent call last)

Cell In[10], line 5

3 seconds = 90*60

4 for remaining_time in tqdm(range(seconds),total=seconds,desc='90 Minute Time

r',unit='sec'):

----> 5 time.sleep(1)

6 print('Time is UP --> GG')

KeyboardInterrupt:

Chapter 1 Exercises — Data Cleaning

Northbridge Community Clinic · ~90 minutes · needs C1_Data_Cleaning.ipynb

Objective → task map

C1's five stated learning objectives, plus the duplicate-row item from its summary checklist, mapped to the tasks that assess them.

#	C1 objective	Assessed by	Marks
1	Load and inspect datasets	Task 1	5
2	Identify and handle missing values	Task 3, Task 4	20
3	Detect and treat outliers	Task 6, Task 7	55
4	Clean and format data properly	Task 2, Task 3, Task 7	55
5	Convert data types appropriately	Task 4, Task 5	25
—	"Check for duplicate rows" (C1 checklist)	Task 2	5

Marks overlap because several tasks assess more than one objective. Total: 100.

Tier	Tasks	What it asks of you
Warm-up	1–3	The C1 workflow on unfamiliar data
Core	4–6	The same tools, on questions C1 did not answer for you
Judgement	7	No new methods. Make a decision and defend it in writing.

Before you start

Dataset: `clinic_visits_raw.csv` — 624 appointment records exported from the booking system of a fictional community clinic. It was exported by three different receptionists and nobody checked it. Everything wrong with it is wrong on purpose.

The toolkit. Every task here is solvable with what C1 taught. You need nothing else:

Job	Tool
Inspect	<code>.shape</code> <code>.dtypes</code> <code>.info()</code> <code>.describe()</code> <code>.isnull().sum()</code> <code>.value_counts()</code> <code>.unique()</code>
Missing	<code>.dropna()</code> <code>.fillna()</code> <code>.mode()</code> <code>.median()</code>
Outliers	<code>.quantile()</code> and the IQR rule, <code>.clip()</code>
Text	<code>.apply()</code> with your own function, <code>.str.strip()</code> <code>.str.lower()</code> <code>.str.title()</code>
Types	<code>pd.to_numeric(errors='coerce')</code> <code>pd.to_datetime()</code> <code>.astype()</code>

Two tools C1 names but does not demonstrate. Both appear on C1's own summary checklist and conversion table, so the syntax is given here and you are expected to use it:

```
df.duplicated().sum()           # how many rows are exact copies of an
earlier row
df.drop_duplicates()           # returns a copy with those rows removed
pd.to_datetime(s, errors='coerce') # unparseable dates become
NaT
pd.to_datetime(s, format='%Y-%m-%d', errors='coerce') # parse ONE known
format
```

Work in order. Each task modifies `df` and the next one assumes it. If a cell fails, restart and run from the top rather than skipping ahead.

A warning that applies to the whole notebook. Every wrong answer available here produces output that runs without error and looks entirely reasonable. Nothing will crash to tell you that you got it wrong. Check your results against the data, not against whether the code executed.

Dataset path used in this notebook: `../data/clinic_visits_raw.csv`

Task 1 — First look

Warm-up · 5 marks · Load and inspect datasets

Load `clinic_visits_raw.csv` and inspect it. Report the shape, the dtypes, and the missing-value count per column.

Then answer two questions in a comment or a markdown cell:

(a) Two columns hold numbers but were loaded as `object` (text). Which two, and what does that tell you before you have looked at a single value?

(b) `patient_age` and `wait_minutes` both have missing values, but `.isnull().sum()` reports missing values for only one of them. Which one is lying to you, and what is the clue that gives it away?

```
In [16]: import pandas as pd
import numpy as np
pd.set_option('display.max_columns', None)
df = pd.read_csv('clinic_visits_raw.csv')
df.isnull().sum()
# Your code here
print(df.shape)
print(df.dtypes)
```

```
(624, 7)
visit_id      object
visit_date    object
department    object
patient_age   float64
wait_minutes  object
patient_weight object
billed_amount float64
dtype: object
```

Your answer to Task 1:

(write here)

Task 2 — Duplicate rows

Warm-up · 5 marks · Clean and format data properly / C1 checklist item

Part of this export was submitted twice.

1. How many rows are exact duplicates of an earlier row?
2. Does `visit_id` agree with that count?
3. Remove them, then report the mean `billed_amount` before and after.
4. Why must this step come before you compute any statistic at all?

```
In [17]: # Your code here
df["visit_id"].duplicated().sum()
df['billed_amount'].mean()
```

```
df_clean = df.drop_duplicates()
df_clean['billed_amount'].mean()
```

Out[17]: np.float64(921.09705)

Why must this step come before you compute any statistic at all ## Answer : Duplicate rows same value counted than once and this skews any statistic (mean ,sum , count) away from the true value ## if you compute first then clean the earlier number is already wrong and cleaning after doesn't retroactively fix it # So dedup must happen before any calculation My own answer: Let me give you an example , like if you have 3 candies and your friend give you a fake candies that is been duplicated , and after received finish you think that total candies got four but it just three , because the fake candies it been duplicated with printed photo so before you want to start count please remove the fake candies first and start your own actual count.

Your answer to Task 2:

(write here)

Task 3 — The department column, and the order of operations

Warm-up · 10 marks · Identify and handle missing values / clean and format

`department` has missing values. C1 taught you to fill a categorical column with its mode. Do exactly that, but in two different orders, and compare.

1. How many distinct values does pandas see in `department` ? How many departments do you think the clinic actually has?
2. **Order A:** take the mode of the column as loaded. Write down which department it is.
3. **Order B:** normalise whitespace and capitalisation first, *then* take the mode. Write down which department it is now.
4. Are they the same department? If not, how many patients would Order A have assigned to the wrong one? Fix `df['department']` using the correct order.

```
In [18]: # Your code here
df['department'].nunique()
```

Out[18]: 14

Your answer to Task 3:

(write here)

```
In [19]: df['department'].unique()
```

```
Out[19]: array(['Orthopaedics', 'Oncology', 'Radiology', 'Dermatology',
               'orthopaedics ', 'Pediatrics', 'CARDIOLOGY', 'Cardiology',
               ' Cardiology', nan, 'cardiology', 'radiology', 'Cardiology ',
               'PEDIATRICS', ' dermatology'], dtype=object)
```

```
In [20]: df['department'].mode()
```

```
Out[20]: 0    Oncology
         Name: department, dtype: object
```

```
In [21]: df['department']=df['department'].str.strip().str.lower()
```

```
In [31]: df['department'].str.strip().str.lower().mode()
```

```
Out[31]: 0    cardiology
         Name: department, dtype: object
```

Task 4 — The missing values that are not missing

Core · 10 marks · Convert data types / handle missing values

In Task 1 you found that `.isnull()` reports zero missing `wait_minutes`.

1. Convert the column with `pd.to_numeric(..., errors='coerce')`. How many values are actually missing?
2. Before you overwrite anything, find out what those non-numeric values *were*. Print them with their counts. (Why does it matter what they say?)
3. Impute the missing wait times. Choose mean or median, and justify the choice in one sentence using a number you have computed, not a rule you have memorised.

```
In [32]: # Your code here
         df['wait_minutes'].isnull
```

```
Out[32]: <bound method Series.isnull of 0          27
         1          72
         2          16
         3    not recorded
         4          19
         ...
        619          18
        620    not recorded
        621          39
        622          33
        623          39
         Name: wait_minutes, Length: 624, dtype: object>
```

```
In [33]: missing_count = pd.to_numeric(df['wait_minutes'],errors= 'coerce').isnull().sum()
         missing_countP
```

```

-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_2200\3779421660.py in ?()
----> 1 missing_count = pd.to_numeric(df['wait_minutes'], errors= 'coerce').isnull().
sum()
      2 missing_count

~\anaconda3\Lib\site-packages\pandas\core\generic.py in ?(self, name)
    6317         and name not in self._accessors
    6318         and self._info_axis._can_hold_identifiers_and_holds_name(name)
    6319     ):
    6320         return self[name]
-> 6321     return object.__getattr__(self, name)

AttributeError: 'Series' object has no attribute 'to_numeric'

```

```

In [ ]: **Your answer to Task 4:**

        *(write here)*

```

Task 5 — Three receptionists, three date formats

Core · 15 marks · Convert data types appropriately

`visit_date` was typed by three different people.

1. Run `pd.to_datetime(df['visit_date'], errors='coerce')` and count how many rows became `NaT`. Before you look: how many do you *expect* to lose?
2. Look at which rows survived and which did not. Explain why that particular group survived. (This is the part that catches people.)
3. Recover **all** rows. State how many are unparseable when handled properly.
4. What is the general lesson about `errors='coerce'` on a date column?

```

In [ ]: # Your code here

```

Your answer to Task 5:

(write here)

Task 6 — A sentinel hiding inside an outlier check

Core · 15 marks · Detect and treat outliers

The billing system writes `9999` into `billed_amount` when a claim is rejected. Nobody told the analytics team.

1. Run `.describe()` on `billed_amount`. What in that output should have caught this?
2. Count the sentinel rows.
3. Run the IQR outlier check **with** the sentinel present. Report the upper bound and the number of rows flagged.
4. Replace the sentinel with `NaN` and run the same check again. Report both numbers.
5. Two separate things went wrong in step 3. Name both. One is obvious; the other is the reason this task is worth 15 marks.

In []: `# Your code here`

Your answer to Task 6:

(write here)

Task 7 — The colleague's clean weight column

Judgement · 40 marks · Detect and treat outliers / clean and format data properly

A colleague cleaned `patient_weight` for you. They copied C1's `clean_length` function, swapped `miles` for the weight units, and got this:

```
def clean_weight(val):
    if pd.isna(val):
        return np.nan
    text = str(val).lower()
    for unit in ('kgs', 'kg', 'lbs', 'lb'):
        text = text.replace(unit, '')
    try:
        return float(text.strip())
    except ValueError:
        return np.nan
```

Their notebook runs without a single error. Their mean patient weight is 79.8 kg, which is a perfectly ordinary figure. Their IQR check flags 50 outliers, so they capped them at the upper bound, exactly as C1 demonstrated, and moved on.

Their column is wrong. Every number they produced is plausible and none of it is right.

1. Reproduce their result: the cleaned column, the mean, the IQR bounds, the 50 flagged rows.
2. Print the **raw strings** behind those 50 flagged rows. What do they have in common that the cleaned numbers no longer show?
3. Produce a correct `patient_weight`, and report the corrected mean and the number of genuine outliers.
4. **Written answer, 100 to 150 words.** State what capping would have done to those 50 patients specifically, why no error message could ever have warned your colleague, and

give a general rule for when IQR capping is safe to apply. Marks are for the reasoning, not for the code.

No new methods are needed. Everything here is in C1.

In []: `# Your code here`

Your answer to Task 7:

(write here)

Before you submit

Work through C1's own summary checklist against your cleaned `df` :

- ☐ Loaded and inspected the data structure
- ☐ Identified and documented missing values (including the ones `.isnull()` missed)
- ☐ Applied an appropriate strategy for each column with missing data
- ☐ Detected outliers using visual and statistical methods
- ☐ Handled outliers appropriately, and can say *why* for each decision
- ☐ Cleaned messy text and mixed-format columns
- ☐ Converted every column to an appropriate data type
- ☐ Checked for and removed duplicate rows

Then run `df.info()` one last time. Every column should have the dtype its meaning implies, and you should be able to explain every remaining missing value.